

Basic Compiler

Project Report

Course: Compilers — Spring 2026

Academic Year: 4th Software Engineering 2025-2026

Submission Date: May 3rd, 2026

Prepared by:

Youssef Fahdloun
Mohamed Ali Guerfali

Iyed Mdimegh
Ahmed Jammoussi

1 Project Overview

This project implements a complete front-end compiler pipeline for a custom programming language, developed in **Python** with a **PyQt5** graphical user interface. The compiler covers the three classical front-end phases, **lexical analysis**, **syntax analysis**, and **semantic analysis**, and exposes each phase interactively through a dedicated tab in the GUI, plus a unified **Pipeline** tab that runs all phases in sequence.

The custom language supports two data types (`int`, `string`), variable declarations, arithmetic and comparison expressions, `if/then/else` conditionals, and `while` loops. Programs are terminated by semicolons and blocks delimited by curly braces.

Lexer
Regex-based tokenizer producing a typed token stream with line/column tracking

Parser
Recursive-descent parser building a typed AST; LR trace simulated from the AST

Semantic Analyzer
Visitor-pattern checker enforcing scope and type correctness via a symbol table

2 Milestone 1 — Lexical Analysis

2.1 Design

The lexer is implemented in `src/lexer.py` as a class `Lexer` that iterates through the source text using a **priority-ordered list of pre-compiled regular expressions** (`TOKEN_PATTERNS`). Patterns are tried left-to-right; the first match wins. This approach cleanly handles keyword disambiguation (e.g., `int` must be matched before the general `IDENTIFIER` pattern).

The `TokenType` enum defines 23 terminal categories:

Category	Tokens
Keywords	<code>KW_IF</code> , <code>KW_THEN</code> , <code>KW_ELSE</code> , <code>KW_WHILE</code> , <code>KW_INT</code> , <code>KW_STRING</code>
Identifiers & Literals	<code>IDENTIFIER</code> , <code>NUMBER</code> , <code>STRING_LITERAL</code>
Arithmetic Operators	<code>PLUS</code> , <code>MINUS</code> , <code>MULTIPLY</code> , <code>DIVIDE</code>
Comparison & Assignment	<code>EQUALS</code> , <code>GREATER</code> , <code>LESS</code>
Delimiters	<code>LPAREN</code> , <code>RPAREN</code> , <code>LBRACE</code> , <code>RBRACE</code> , <code>SEMICOLON</code>
Control	<code>EOF</code> , <code>UNKNOWN</code>

Each produced `Token` carries its `type`, `value`, and precise `(line, column)` position, enabling accurate downstream error reporting. Whitespace tokens are silently consumed. An `EOF` sentinel is appended at the end of every token stream.

2.2 Example

For input `int x; x = 10;`, the lexer produces: `KW('int')` `ID('x')` `';'` `ID('x')` `'='` `NUM(10)` `';'` `EOF`

The GUI's **Lexer** tab displays the full token stream in a sortable table showing line, column, type, and value for every token, providing immediate feedback on tokenization accuracy (see Figure 1).

Basic Compiler																																												
Examples: While loop																																												
Lexer Parser Semantic analyzer Pipeline																																												
Source code <pre> int x; x = 10; while x > 0 { x = x - 1; } </pre>																																												
Tokenize																																												
Token stream <table> <tr> <th></th><th>Line</th><th>Column</th><th>Type</th><th>Value</th></tr> <tr> <td>1</td><td>1</td><td>1</td><td>KW_INT</td><td>int</td></tr> <tr> <td>2</td><td>1</td><td>5</td><td>IDENTIFIER</td><td>x</td></tr> <tr> <td>3</td><td>1</td><td>6</td><td>SEMICOLON</td><td>;</td></tr> <tr> <td>4</td><td>2</td><td>1</td><td>IDENTIFIER</td><td>x</td></tr> <tr> <td>5</td><td>2</td><td>3</td><td>EQUALS</td><td>=</td></tr> <tr> <td>6</td><td>2</td><td>5</td><td>NUMBER</td><td>10</td></tr> <tr> <td>7</td><td>2</td><td>7</td><td>SEMICOLON</td><td>;</td></tr> </table>						Line	Column	Type	Value	1	1	1	KW_INT	int	2	1	5	IDENTIFIER	x	3	1	6	SEMICOLON	;	4	2	1	IDENTIFIER	x	5	2	3	EQUALS	=	6	2	5	NUMBER	10	7	2	7	SEMICOLON	;
	Line	Column	Type	Value																																								
1	1	1	KW_INT	int																																								
2	1	5	IDENTIFIER	x																																								
3	1	6	SEMICOLON	;																																								
4	2	1	IDENTIFIER	x																																								
5	2	3	EQUALS	=																																								
6	2	5	NUMBER	10																																								
7	2	7	SEMICOLON	;																																								

Figure 1: Lexer tab — token stream for a while-loop program

3 Milestone 2 — Syntax Analysis

3.1 Formal Grammar

The language grammar, defined in `docs/grammar.md`, is an unambiguous context-free grammar with standard arithmetic precedence enforced structurally: `::= ::= | | | ::= ('int' | 'string') ; ::= = ; ::= 'if' 'then' ['else'] ::= 'while' '{ '}' ::= (('>' | '<' | '=') ) ::= (('+' | '-') ) ::= (('' | '/') ) ::= '-' | ::= | | | '(' '('`

Operator precedence is encoded directly in the grammar hierarchy: comparison → addition → multiplication → unary → primary, matching standard mathematical conventions.

3.2 Recursive-Descent Parser

The parser (`src/parser.py`) is a hand-written **recursive-descent** parser. Each non-terminal in the grammar corresponds to a dedicated `parse_*` method. Key design choices:

- `expect(token_type)` consumes the expected token or logs an error and returns `None`, allowing parsing to continue and collect multiple errors in a single pass.
- `parse_if_statement` supports both braced blocks `{ ... }` and single inline statements after `then / else`, giving flexible syntax.
- `parse_expression` → `parse_comparison` → `parse_addition` → `parse_multiplication` → `parse_unary` → `parse_primary` implements a clean recursive chain respecting full operator precedence including parenthesised sub-expressions.

3.3 Abstract Syntax Tree

All AST node classes live in `src/ast_nodes.py` and inherit from a common `ASTNode` base. Every node stores its source position (`line`, `column`) and exposes an `accept(visitor)` method implementing the **Visitor pattern**, which decouples tree traversal from node structure. Node types include: `Program`, `Declaration`, `Assignment`, `IfStatement`, `WhileStatement`, `BinaryExpression`, `UnaryExpression`, `Identifier`, `NumberLiteral`, `StringLiteral`, and `EmptyStatement`.

3.4 Parser GUI Views

The Parser tab offers three sub-views to interact with the structural output:

- **Top-down tree:** a collapsible textual outline of the AST hierarchy, making it easy to drill down into specific blocks, as visualized in Figure 2.

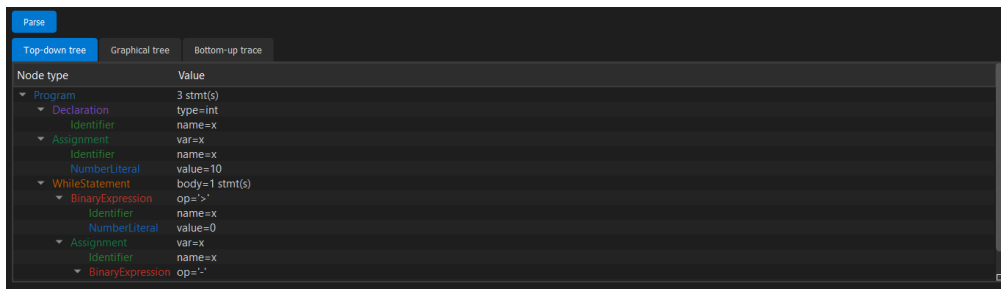


Figure 2: Top-down hierarchical text-based view of the AST

- **Graphical tree:** a zoomable, pannable canvas rendering the AST with colour-coded node types (`ast_canvas_widget.py`), drawn with a recursive layout algorithm to decouple visual evaluation from raw structure (Figure 3).

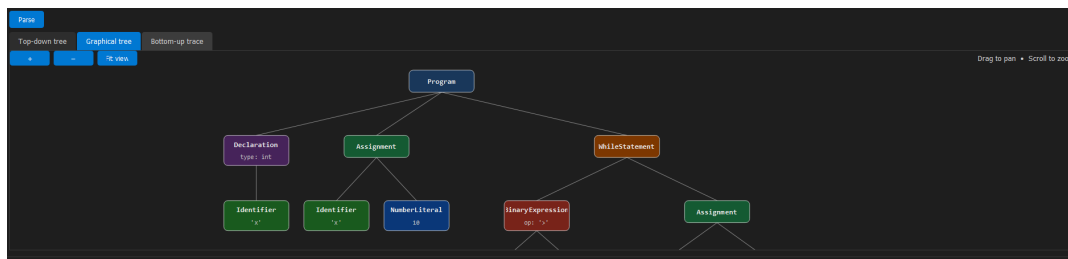


Figure 3: Interactive Graphical AST Tree rendering colour-coded nodes

3.5 Bottom-Up Trace

Because an LR trace is pedagogically valuable, `src/lr_parser_trace.py` implements a post-hoc **shift-reduce simulation**: it walks the finished AST and emits one **SHIFT** action per leaf token and one **REDUCE** action per interior node, annotated with the corresponding grammar rule.

- **Bottom-up trace view:** a step-by-step animation of the shift-reduce sequence with Prev / Next / Auto-play controls and adjustable speed. This produces an accurate bottom-up derivation trace (Figure 4) without requiring an actual LR parser engine.

Step	Action	Symbol	Stack	Lookahead	Rule / Note
29	REDUCE	stmt	int primary stmt... }	stmt	→ IDENTIFIER = expr ;
30	SHIFT	}	int primary stmt... \$		
31	REDUCE	stmt	int primary stmt... \$	stmt	→ while expr { block }
32	REDUCE	program	int primary stmt... \$	program	→ statement*
33	ACCEPT	\$	int primary stmt... \$		Input accepted

Figure 4: Bottom-up trace — final steps showing REDUCE and ACCEPT actions for the while-loop program

4 Milestone 3 — Semantic Analysis

4.1 Symbol Table

`src/symbol_table.py` implements a **scoped symbol table** as a stack of dictionaries (`self.scopes`). Operations:

- `enter_scope()` / `exit_scope()`: push/pop a new scope frame; called at every block boundary (`if`, `while`).
- `declare(name, type, line, col)`: inserts a symbol in the current scope; returns an error if already declared.
- `lookup(name)`: searches scopes from innermost to outermost, supporting lexical scoping.
- `get_type(name)`: convenience wrapper returning the declared type string (`"int"` or `"string"`).

4.2 Semantic Analyzer

`src/semantic_analyzer.py` implements the `SemanticAnalyzer` class as a **visitor** over the AST. It traverses every node and enforces three categories of rules:

Check	Description	Example Error
Scope	Variable used before declaration	Use of undeclared variable: z
Duplicate	Variable declared twice in same scope	Variable 'x' already declared
Type mismatch	Assignment of incompatible type	Cannot assign string to int
Expression types	Arithmetic on strings	Cannot perform arithmetic on strings

The `visit_binary_expression` method is the core of type inference: it recurses on both sub-expressions, retrieves their types, and enforces rules. For instance, `+` is permitted between two `string` values (concatenation) but raises a semantic error if one operand is `int` and the other `string`. Comparison operators always return `"int"` (treated as boolean). Unary negation propagates the operand's type.

The Semantic tab of the GUI displays both a **symbol table** (scope, name, declared type, position) and a **semantic errors** panel with precise line/column information. Figure 5 illustrates how the analyzer proactively captures scoping and typing infractions dynamically.

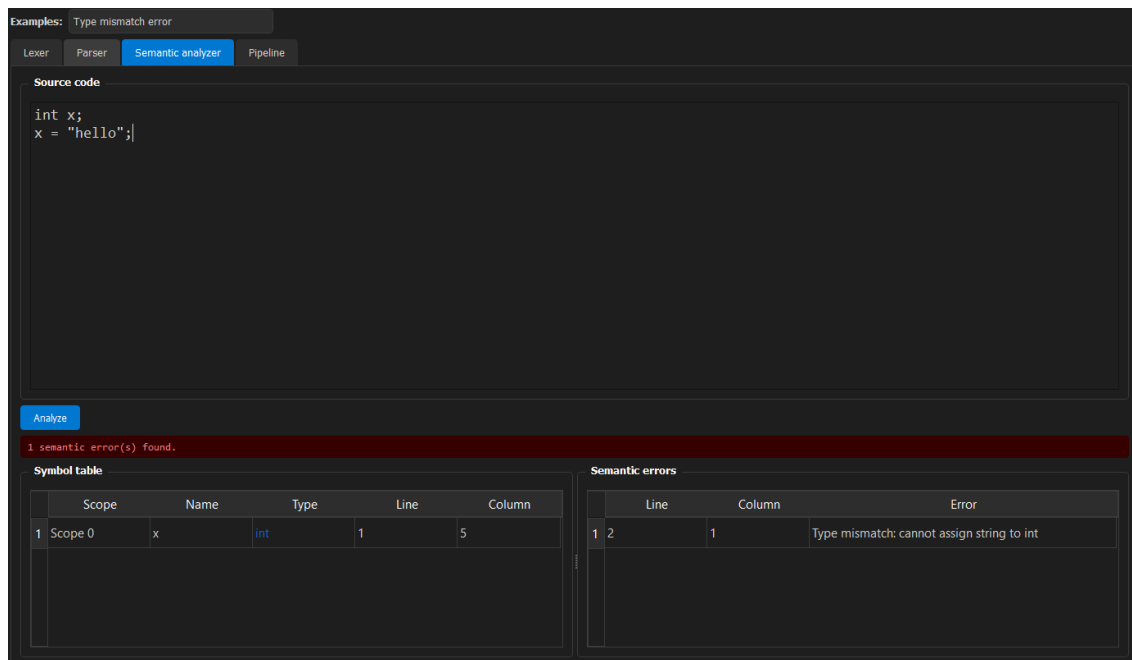


Figure 5: Semantic Analyzer successfully catching a Type Mismatch and displaying Symbol Table state

5 Pipeline & GUI Architecture

The Pipeline tab (`gui/tabs/pipeline_tab.py`) chains all three phases in sequence via the `Compiler` facade class (`src/compiler.py`). A shared `ErrorHandler` (`src/error_handler.py`) collects lexer, parser, and semantic errors independently; the pipeline reports success or the first failing phase.

As shown in Figure 6, the GUI displays each phase (Lexical analysis → Syntax analysis → Semantic analysis → Compilation complete) with a **SUCCESS** / **ERROR** badge and a detail line (e.g., “Generated 32 tokens”, “Built AST with 5 statement(s)”, “Checked 2 symbol(s) across 1 scope(s)”).

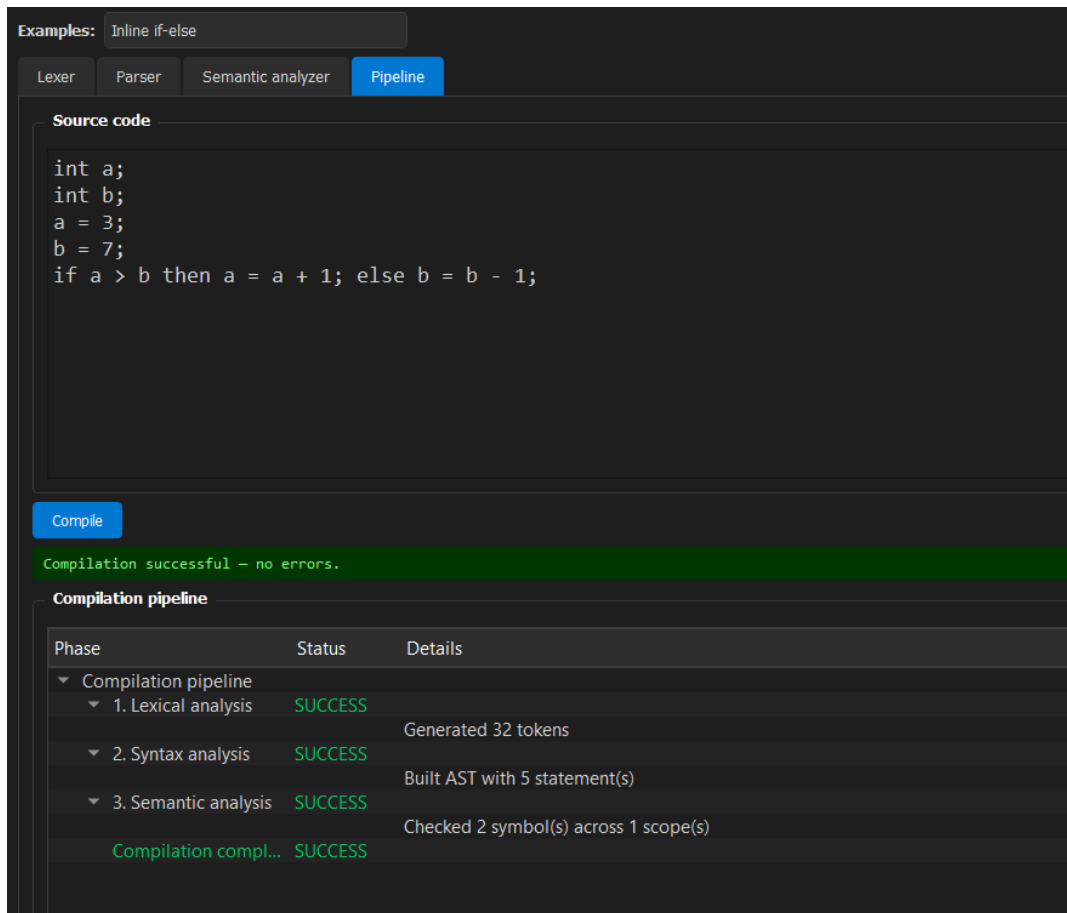


Figure 6: Successful compilation pipeline run through all 3 distinct phases

Additional GUI components include:

- `gui/syntax_highlighter.py`: real-time syntax highlighting in the source editor using `QSyntaxHighlighter` with regex rules for keywords, strings, numbers, and operators.
- `gui/widgets/ast_canvas_widget.py`: interactive graphical AST canvas with zoom and pan, auto-layout, and colour-coded node types.
- `gui/widgets/bottom_up_widget.py`: animated bottom-up trace table with playback controls.
- `gui/widgets/symbol_table_widget.py`: tabular symbol table display.
- `gui/widgets/parse_tree_widget.py`: collapsible top-down tree view.

Several pre-loaded **example programs** ship with the application (while loop, inline if-else, type mismatch, calculator, conditional logic), selectable from a dropdown to immediately populate the editor.

6 Testing

`tests/test_compiler.py` contains a comprehensive automated test suite covering the full pipeline. Test programs are stored as `.src` files under `tests/test_programs/` in two categories:

- **Valid programs:** `arithmetic.src`, `basic_assignment.src`, `conditional.src`, `string_ops.src`: verify that valid code produces zero errors and a well-formed AST.
- **Invalid programs:** `type_mismatch.src`, `undeclared_var.src`: verify that specific semantic errors are raised with correct messages.

The tests exercise the `Compiler` class end-to-end, asserting both the absence of errors for correct input and the presence of expected error messages for erroneous input.

All source files available in this repository:
<https://github.com/ajammoussi/basic-compiler>